

TD4 Pipes

Exercise 1

Write a program that determines the capacity of the buffer (allocated by the Linux system) for a communication pipe.

Exercise 2

Write a program that creates two processes. The parent process opens a text file for reading. It is assumed that the file is composed of words formed of alphabetic characters separated by spaces. The child process enters a word on the keyboard. The parent process searches for the word in the file, and passes the value 1 to the child if the word is in the file, and 0 otherwise. The child displays the result.

```
char * strtok ( char * str, const char * delimiters );
```

Exercise 3

What are the displays produced when running this program?

```
#include<stdio.h>
#include<stdlib.h>
#include<signal.h>
#include<string.h>
int main(){
char buf[1];
int tube[2], tube2[2];
pipe(tube2);
pipe(tube);
switch (fork ()){
case -1: perror("fork"); exit(-1);
case 0: {
read(tube[0],buf,1);
printf("ca\n");
write(tube2[1],"1",1);
exit(0);
}
default:
printf("wow\n");
write(tube[1],"1",1);
read(tube2[0],buf,1);
printf("compile\n");
wait(NULL);
return 0;
}
}
```

We want to rewrite this program, without using tubes, ensuring the same synchronization of the processes. The new program should produce the same displays.

Edit the gray parts of this code to implement the same synchronization with

1. the sleep function
2. the wait wire termination function
3. signals, for example, SIGUSR1

Exercise 4

N people play with a bomb set off with a countdown timer. Each person holds the bomb for a while and then passes it to their neighbor, the bomb continues to pass circularly from person to person until it reaches the 0:00 counter when it explodes. The last person who possessed the bomb before it exploded is hit and is declared the loser of the game. Program this game in Unix, based on the notions acquired on processes, signals and communication tubes. The

number of people involved in the game, and the bomb explosion delay are input parameters to the program.

Exercise 5

a) What happens while the next program is running? <pre> 1 # include < stdlib .h> 2 # include < stdio .h> 3 # include < unistd .h> 4 # include < sys / wait .h> 5 int p [2]; 6 void fils1 (void) 7 { 8 char c; 9 close (p [0]); 10 printf (" d'ebut fils1 (taper 0 pour finir)\n"); 11 while ((c= getchar ()) != '0 ') 12 if ((c >= 'A ' && c <= 'Z ') (c >= 'a ' && c <= 'z ')) 13 { 14 if ((c >= 'a ') && (c <= 'z ')) c -=32; 15 write (p[1] ,&c ,1); 16 printf (" Le fils1 envoie >%c <\n",c); 17 } 18 close (p [1]); 19 exit (EXIT_SUCCESS); 20 } 21 void fils2 (void) 22 { 23 char c; 24 close (p [1]); 25 printf (" debut fils2 \n"); 26 while (read (p[0] ,&c ,1) > 0) 27 printf (" fils2 re,coit >%c <\n",c); 28 close (p [0]); 29 exit (EXIT_SUCCESS); 30 } 31 int main (void) 32 { 33 if (pipe (p) != 0) 34 { 35 printf (" pb ouverture pipe \n"); 36 exit (EXIT_FAILURE); 37 } 38 if (fork ()==0) fils1 (); 39 if (fork ()==0) fils2 (); 40 close (p [0]); close (p [1]); 41 wait (NULL); wait (NULL); 42 printf (" fin du pere \n"); 43 exit (EXIT_SUCCESS); 44 } </pre>	b) What happens while the following tubonacci.c program is executed?1 # <pre> include < stdlib .h> 2 # include < stdio .h> 3 # include < unistd .h> 4 # include < wait .h> 5 void parent (int entree , int sortie) 6 { 7 unsigned char n = 0; 8 write (sortie , &n , 1); 9 n++; write (sortie , &n , 1); 10 do { 11 read (entree , &n , 1); 12 printf ("%u " , n); fflush (stdout); 13 write (sortie , &n , 1); 14 } while (n <= 200); 15 printf ("\n"); 16 close (sortie); 17 wait (NULL); 18 } 19 void enfant (int entree , int sortie) 20 { 21 unsigned char p,q; 22 read (entree , &p , 1); 23 for (;){ 24 write (sortie , &p , 1); 25 if (read (entree , &q , 1) != 1) return ; 26 p += q; 27 } 28 } 29 int main (void) 30 { 31 int tube_pe [2] , tube_ep [2] , pid; 32 printf ("%d] : d'ebut du p`ere .\n", getpid ()); 33 pipe (tube_pe); /* tube Parent -> Enfant */ 34 pipe (tube_ep); /* tube Enfant -> Parent */ 35 if ((pid = fork ()) == -1) exit (EXIT_FAILURE); 36 if (pid > 0){ /* parent */ 37 close (tube_pe [0]); close (tube_ep [1]); 38 parent (tube_pe [0] , tube_ep [1]); 39 } 40 else { /* enfant */ 41 close (tube_pe [1]); close (tube_ep [0]); 42 enfant (tube_pe [0] , tube_ep [1]); 43 } 44 printf ("%d] : terminaison \n", getpid ()); 45 exit (EXIT_SUCCESS); 46 } </pre> c) Que ce passe-t-il si l'on supprime la ligne 19 de tubonacci.c ? d) Modifier tubonacci.c pour calculer les termes de la suite définie par $u_0 = 2$, $u_1 = 3$, $u_{n+2} = 2u_{n+1} + 3u_n$.
--	--

Exercise 6

Error hunting. What should the following program do? Find at least two mistakes and correct them.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <signal.h>
5
6 static int n = 0;
7
8 void erreur ( const char * errmsg )
9 {
10 perror (errmsg); exit ( EXIT_FAILURE );
11 }
12
13 void interruption (int signum )
14 {
15 printf ("No choisi : % d\n",n); exit ( EXIT_SUCCESS );
16 }
17
18 void init_lecture (int d_lecture )
19 {
20 write ( d_lecture ,&n, sizeof ( int ));
21 }
22
23 void iteration (int d_lecture , int d_écriture )
24 {
25 while (1){
26 read ( d_lecture ,&n, sizeof (int ));
27 n++;
28 write ( d_écriture ,&n, sizeof ( int ));
29 }
30 }
31
32 int main ( void )
33 {
34 int pf [2] , fp [2]; /* Communication bidirectionnelle */
35 struct sigaction action ;
36 action . sa_flags =0;
37 sigemptyset (& action . sa_mask );
38
39 if( pipe (pf ) == -1 || pipe (fp ) == -1 ) erreur (" creation d'un tube ");
40 switch ( fork ()){
41 case -1: erreur (" fork ");
42 case 0 : /* Fils */
43 close (fp [0]); close (pf [1]);
44 iteration (pf [0] , fp [1]);
45 default : /* Pere */
46 action . sa_handler = interruption ;
47 sigaction (SIGINT,& action , NULL );
48 close (pf [0]); close (fp [1]);
49 iteration (fp [0] , pf [1]);
50 }
51 exit ( EXIT_FAILURE );
52 }
```

Exercise 7

Write a C program that results in 2 processes executing where a parent process creates one child process. The parent reads from the user two integer numbers a and b, and sends their values to the child.

The child computes *the greatest common divisor GCD* and display the result on the screen. The inter-processes communication should be establish using pipes.

Sample run

Enter a b: 21 9

The GCD of 21 and 9 is 3

Exercise 8

Write a program that takes two integers from the command line arguments and reports the number with the greater number of divisors. The program first sets a pipe for inter-process communication. It then forks a new process where:

The child calculates the number of divisors of `argv[2]` and writes it to the pipe.

The parent calculates the number of divisors of `argv[1]`, reads the number written by the child in the pipe, compares the two counts and reports the bigger one,

Use the `atoi(char*)` function to convert the string to an integer.

Home practice: allow the user to enter an unknown number of integers on the command line. For each integer, the program forks a child to calculate its number of divisors and send it back to the parent through a pipe.